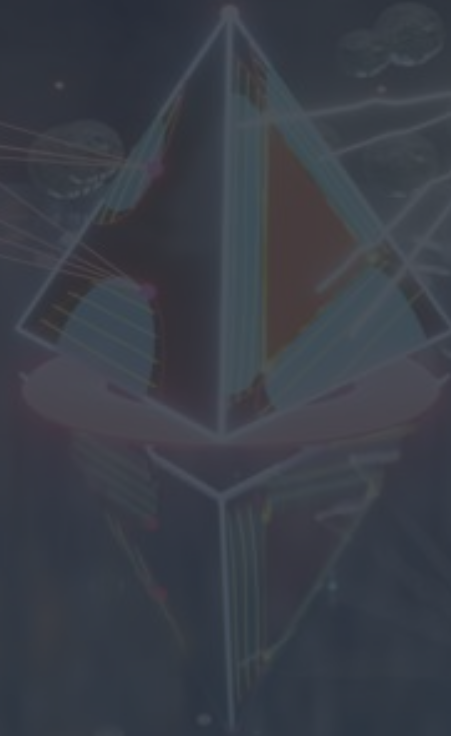
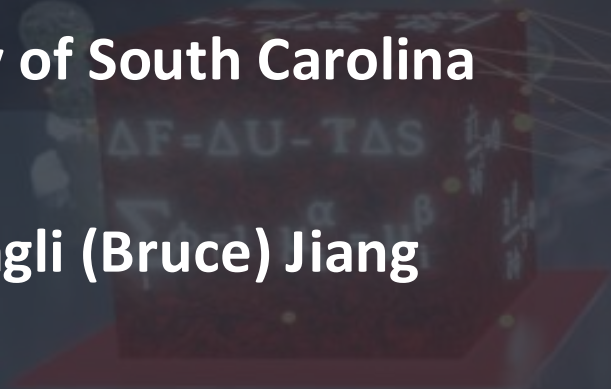


Regression Evaluation and Nonlinear Models

Fall 2026

University of South Carolina

Prof. Shengli (Bruce) Jiang



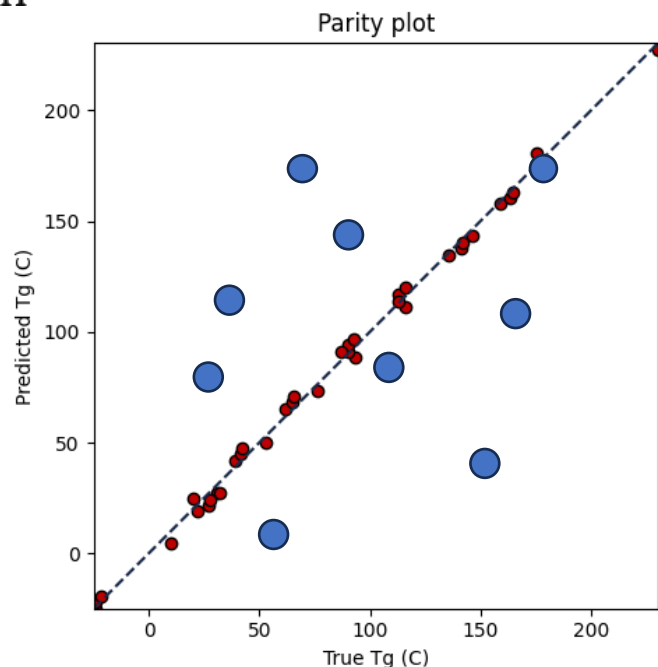
Training Error vs. Test Error

Training Data: Used to learn the model parameters

```
from sklearn.linear_model import LinearRegression  
  
model = LinearRegression()  
model.fit(X, y)
```

$$\frac{1}{n_{\text{train}}} \sum (y_i - \hat{y}_i)^2$$

How well does the model predict data it has already seen?



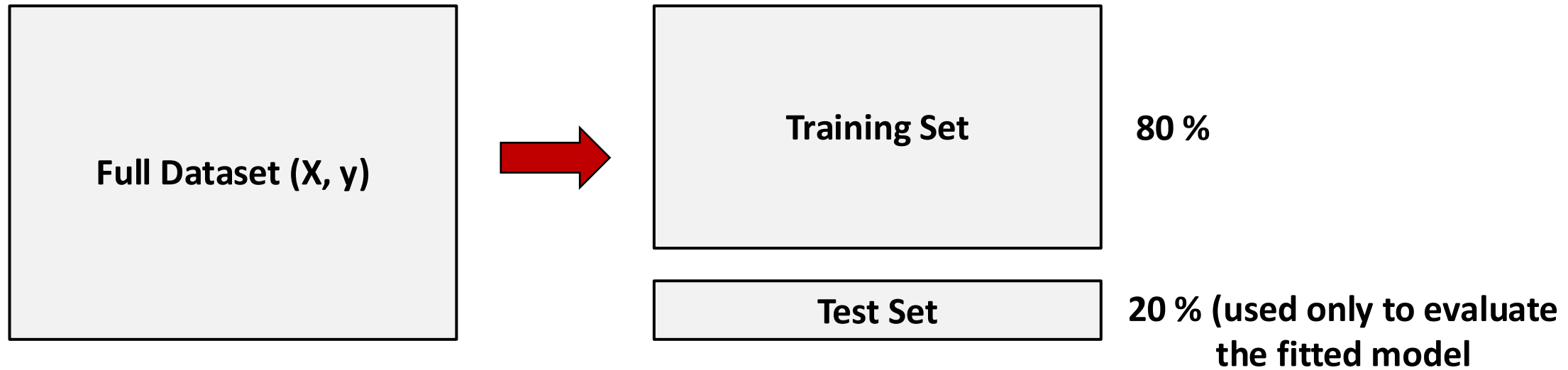
Test Data: Evaluate model performance. **Not used during model fitting/training!**

$$\frac{1}{n_{\text{test}}} \sum (y_i - \hat{y}_i)^2$$

How well does the fitted model predict new data?

- A low training error is not enough.
- A useful model must also have a low test error.

Train–Test Split



train_test_split

```
sklearn.model_selection.train_test_split(*arrays, test_size=None,  
train_size=None, random_state=None, shuffle=True, stratify=None)
```

Split arrays or matrices into random train and test subsets.

[\[source\]](#)

Quick utility that wraps input validation, `next(ShuffleSplit().split(X, y))`, and application to input data into a single call for splitting (and optionally subsampling) data into a one-liner.

Train–Test Split Parameters

```
import numpy as np
from sklearn.model_selection import train_test_split
```

```
X = np.arange(10).reshape((5, 2))
y = np.arange(5)
```

```
print(X)
print(y)
```

```
[[0 1]
 [2 3]
 [4 5]
 [6 7]
 [8 9]]
[0 1 2 3 4]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=26, shuffle=True)
```

```
print(X_train)
print(X_test)
```

```
[[6 7]
 [4 5]
 [2 3]
 [0 1]]
[[8 9]]
```

```
print(y_train)
print(y_test)
```

```
[3 2 1 0]
[4]
```

X: input features

y: output labels

test_size=0.2: 20% for testing

random_state=26: repeatable split

shuffle=True: randomize samples

Pay attention to the output order!

Effect of Shuffling

```
import numpy as np
from sklearn.model_selection import train_test_split

X = np.arange(10).reshape((5, 2))
y = np.arange(5)

print(X)
print(y)
```

[[0 1]
[2 3]
[4 5]
[6 7]
[8 9]]
[0 1 2 3 4]

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=26, shuffle=False)

print(X_train)
print(X_test)
```

[[0 1]
[2 3]
[4 5]
[6 7]]
[[8 9]]

```
print(y_train)
print(y_test)
```

[0 1 2 3]
[4]

MAE and MSE

<https://scikit-learn.org/stable/api/sklearn.metrics.html>

Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- Typical absolute prediction error
- Same units as the target

```
from sklearn.metrics import mean_absolute_error
y_true = [3, -0.5, 2, 7]
y_pred = [2.5, 0.0, 2, 8]
print(mean_absolute_error(y_true, y_pred))
```

0.5

Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- Give more weight to large errors
- Squared units of the target.

```
from sklearn.metrics import mean_squared_error
y_true = [3, -0.5, 2, 7]
y_pred = [2.5, 0.0, 2, 8]
mean_squared_error(y_true, y_pred)
```

0.375

Root Mean Squared Error

<https://scikit-learn.org/stable/api/sklearn.metrics.html>

Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- Typical absolute prediction error
- Same units as the target

```
from sklearn.metrics import mean_absolute_error
y_true = [3, -0.5, 2, 7]
y_pred = [2.5, 0.0, 2, 8]
print(mean_absolute_error(y_true, y_pred))

0.5
```

Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- Give more weight to large errors
- Different units as the target

Root Mean Squared Error (RMSE)

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

- Same units as the target
- **Only available on some sklearn versions**

Coefficient of Determination (R^2)

coefficient of determination (R^2)

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

- Compares the model with predicting the mean
- Dimensionless

```
from sklearn.metrics import r2_score
y_true = [3, -0.5, 2, 7]
y_pred = [2.5, 0.0, 2, 8]
print(r2_score(y_true, y_pred))
```

0.9486081370449679

R^2 measures how much better the model predicts than simply guessing the average.

$$\bar{y} = \frac{3 + (-0.5) + 2 + 7}{4} = 2.875$$

$$\begin{aligned} \text{SSE} &= \sum_{i=1}^4 (y_i - \hat{y}_i)^2 && \text{Sum of Squared Errors:} \\ &&& \text{prediction error from the model} \\ &= (3 - 2.5)^2 + (-0.5 - 0)^2 + (2 - 2)^2 + (7 - 8)^2 \\ &= 0.25 + 0.25 + 0 + 1 \\ &= 1.5 \end{aligned}$$

$$\begin{aligned} \text{SST} &= \sum_{i=1}^4 (y_i - \bar{y})^2 && \text{Total Sum of Squares: total variation in} \\ &&& \text{the actual values around their mean} \\ &= (3 - 2.875)^2 + (-0.5 - 2.875)^2 \\ &\quad + (2 - 2.875)^2 + (7 - 2.875)^2 \\ &= 29.1875 \end{aligned}$$

$$\begin{aligned} R^2 &= 1 - \frac{\text{SSE}}{\text{SST}} && \text{SSE/SST is the fraction of variation} \\ &= 1 - \frac{1.5}{29.1875} && \text{the model fails to explain.} \\ &= 0.9486 \end{aligned}$$

Limitations of R^2

- **A high R^2 is not sufficient, and a low R^2 is not always useless.**
- R^2 measures improvement over predicting the mean. It does not directly measure acceptable error, bias, or generalization.

```
y_true = [0,100,200,300]
y_pred = [10,110,210,310]
print(r2_score(y_true, y_pred))
print(mean_absolute_error(y_true, y_pred))
```

0.992
10.0

- Every prediction is 10 units too high.
- The model captures the overall variation but has systematic bias.
- It may be unacceptable if the allowed error is only 2 units.

```
y_true = [99,100,101,100]
y_pred = [100,100,100,100]
print(r2_score(y_true, y_pred))
print(mean_absolute_error(y_true, y_pred))
```

0.0
0.5

- The model does not explain variation around the mean.
- However, an average error of 0.5 units may still be acceptable for some applications.

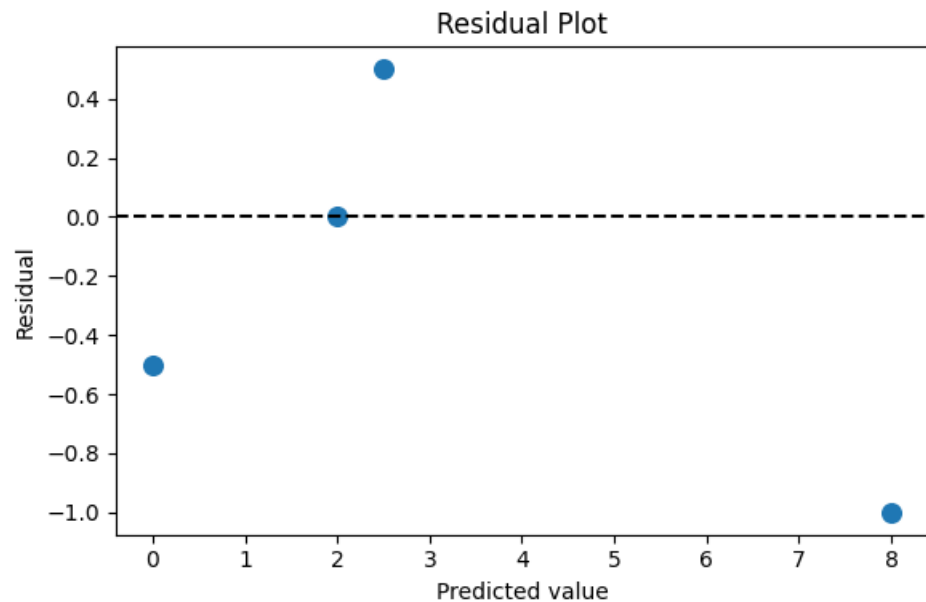
- **Never evaluate a regression model using R^2 alone.**
- Use R^2 together with MAE or RMSE, residual plots, and test data.

MAPE and Residual Plots

Mean Absolute Percentage Error (MAPE)

$$\text{MAPE} = \frac{100\%}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

A metric tells us how large the errors are. A residual plot shows us where and how the errors occur.



```
import numpy as np
import matplotlib.pyplot as plt

y_true = np.array([3, -0.5, 2, 7])
y_pred = np.array([2.5, 0, 2, 8])

residuals = y_true - y_pred

fig, ax = plt.subplots(figsize=(6, 4))

ax.scatter(y_pred, residuals, s=70)
ax.axhline(y=0, color="black", linestyle="--")

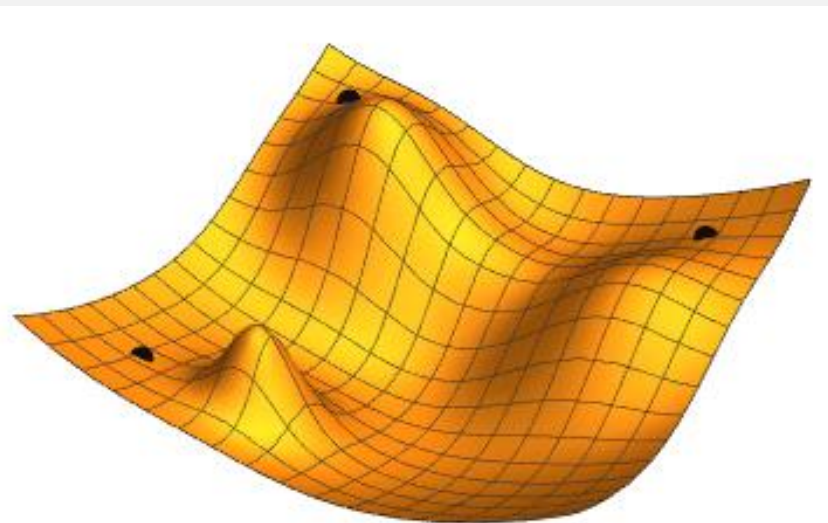
ax.set_xlabel("Predicted value")
ax.set_ylabel("Residual")
ax.set_title("Residual Plot")

fig.tight_layout()
plt.show()
```

Gradient Descent for Least Squares

Gradient Descent

An iterative method that minimizes a function by moving in the direction of steepest decrease.



Very common in neural network training.

Intuition is like taking repeated downhill steps on the error surface.

Step 1: Start with an initial guess

$$\boldsymbol{\theta}^{(0)}$$

Step 2: At each iteration, calculate predictions

$$\hat{\mathbf{y}}^{(t)} = \mathbf{X}\boldsymbol{\theta}^{(t)}$$

Step 3: Calculate residuals

$$\mathbf{e}^{(t)} = \hat{\mathbf{y}}^{(t)} - \mathbf{y}$$

- *very simple method*
- *slowly convergent to closest minima*
- *depends on (adaptive) stepsize*

Step 4: Calculate the gradient

$$J(\boldsymbol{\theta}) = \frac{1}{2n} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2$$

$$\nabla_{\boldsymbol{\theta}} J = \frac{1}{n} \mathbf{X}^T (\mathbf{X}\boldsymbol{\theta}^{(t)} - \mathbf{y})$$

Step 5: Update the parameters

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \nabla_{\boldsymbol{\theta}} J$$

One Gradient Descent Iteration

$$\mathbf{X} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 2 \\ 4 \end{bmatrix}$$

Step 1: Start with an initial guess

$$\boldsymbol{\theta}^{(0)} = [0], \quad \eta = 0.1$$

Step 2: At each iteration, calculate predictions

$$\hat{\mathbf{y}}^{(0)} = \mathbf{X}\boldsymbol{\theta}^{(0)} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} (0) = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Step 3: Calculate residuals

$$\mathbf{e}^{(0)} = \hat{\mathbf{y}}^{(0)} - \mathbf{y} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - \begin{bmatrix} 2 \\ 4 \end{bmatrix} = \begin{bmatrix} -2 \\ -4 \end{bmatrix}$$

Step 4: Calculate the gradient

$$\begin{aligned} \nabla_{\boldsymbol{\theta}} J &= \frac{1}{n} \mathbf{X}^T \mathbf{e}^{(0)} \\ &= \frac{1}{2} \begin{bmatrix} 1 & 2 \end{bmatrix} \begin{bmatrix} -2 \\ -4 \end{bmatrix} \\ &= -5 \end{aligned}$$

Step 5: Update the parameters

$$\begin{aligned} \boldsymbol{\theta}^{(1)} &= \boldsymbol{\theta}^{(0)} - \eta \nabla_{\boldsymbol{\theta}} J \\ &= 0 - 0.1(-5) \\ &= 0.5 \end{aligned}$$

Step 2 again for iteration 1

$$\hat{\mathbf{y}}^{(1)} = \mathbf{X}\boldsymbol{\theta}^{(1)} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} (0.5) = \begin{bmatrix} 0.5 \\ 1.0 \end{bmatrix}$$

Implementing Gradient Descent

```
def gradient_descent(X, y, theta_initial, learning_rate=0.1,
                    n_iterations=10):
    """Fit a linear model using batch gradient descent."""

    X = np.asarray(X, dtype=float)
    y = np.asarray(y, dtype=float)
    theta = np.asarray(theta_initial, dtype=float).copy()

    theta_history = [theta.copy()]
    y_pred = X @ theta
    loss_history = [0.5 * np.mean((y_pred - y) ** 2)]

    for _ in range(n_iterations):
        y_pred = X @ theta

        residuals = y_pred - y

        gradient = X.T @ residuals / len(y)

        theta = theta - learning_rate * gradient

        theta_history.append(theta.copy())

        y_pred = X @ theta
        loss = 0.5 * np.mean((y_pred - y) ** 2)
        loss_history.append(loss)

    return (
        theta,
        np.array(theta_history),
        np.array(loss_history),
    )
```

```
X = np.array([[1], [2]])

y = np.array([2, 4])

theta_initial = np.array([0])

theta, theta_history, loss_history = gradient_descent(
    X,
    y,
    theta_initial,
    learning_rate=0.1,
    n_iterations=10,
)

print("Initial theta", theta_history[0])
print("Theta after one step", theta_history[1])
print("Initial loss", loss_history[0])
print("Loss after one step", loss_history[1])
print("Final theta", theta)
```

Initial theta [0.]
Theta after one step [0.5]
Initial loss 5.0
Loss after one step 2.8125
Final theta [1.88737297]

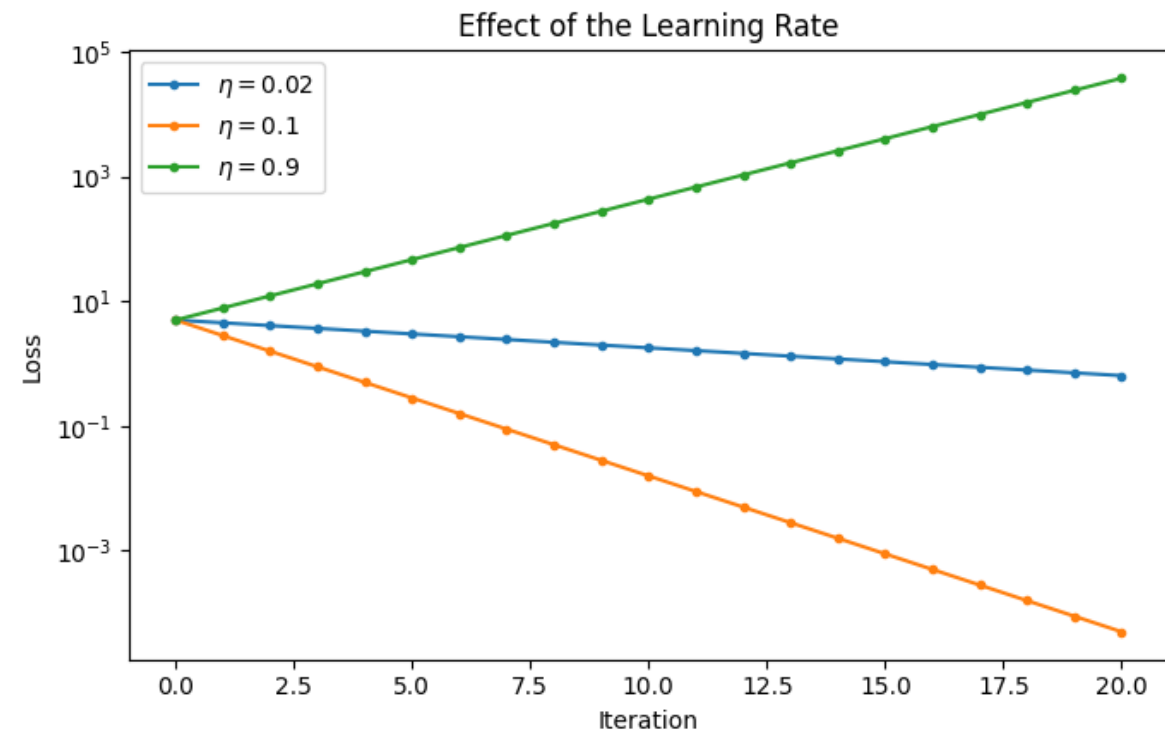
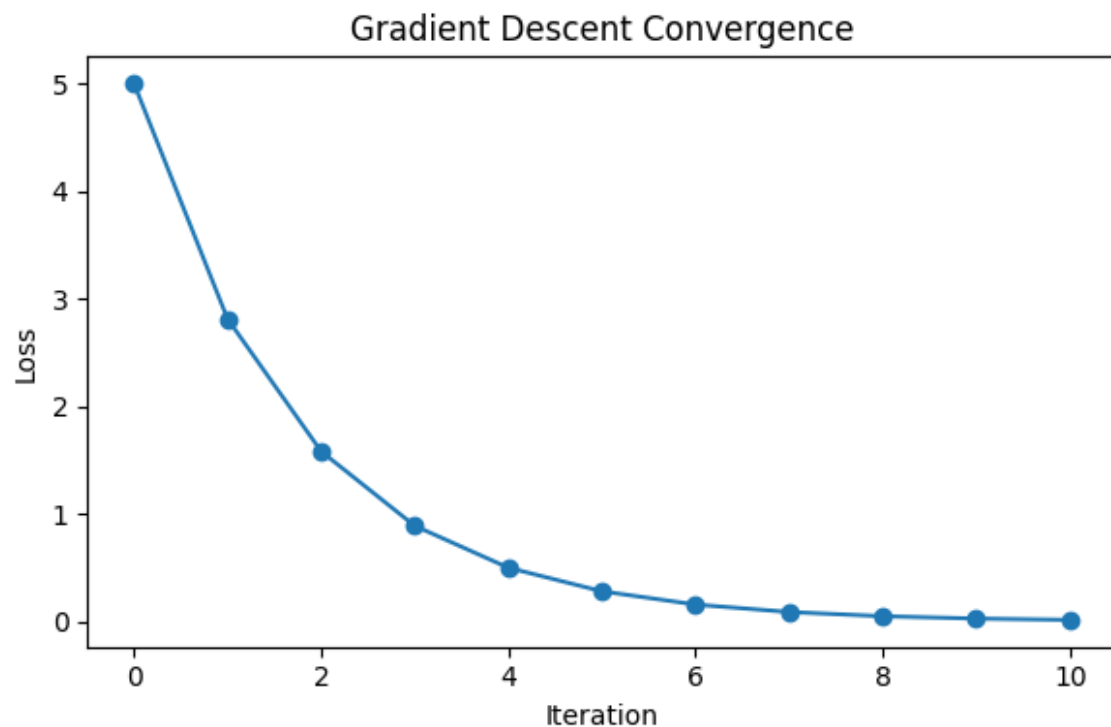
Impact of the Learning Rate

```
fig, ax = plt.subplots(figsize=(6, 4))

iterations = np.arange(len(loss_history))

ax.plot(iterations, loss_history, marker="o")
ax.set_xlabel("Iteration")
ax.set_ylabel("Loss")
ax.set_title("Gradient Descent Convergence")

fig.tight_layout()
plt.show()
```



- $\eta = 0.02$
Loss decreases slowly
- $\eta = 0.1$
Loss decreases quickly and smoothly (Best)
- $\eta = 0.9$
Loss oscillates or increases (**This is called "divergence" or "exploding loss"**)

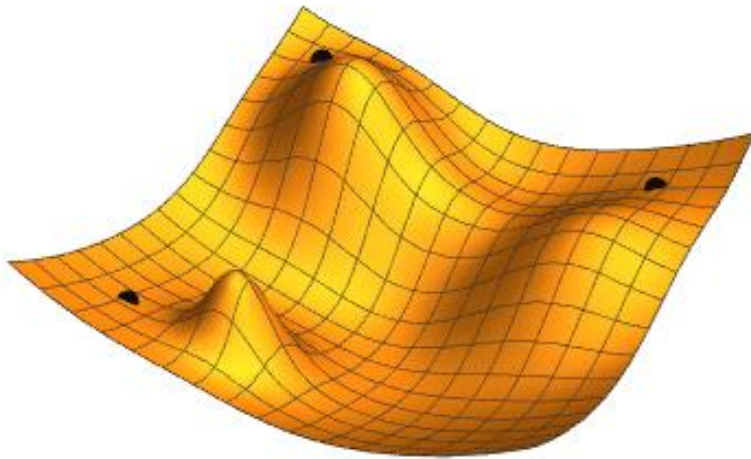
Nonlinear Least Squares

Non-linear regression

$f(x, \boldsymbol{\theta})$ *now just some general function, which is not necessarily linear in its parameters*

e.g.,

$$f(x, \boldsymbol{\theta}) = \theta_0 \cos(\theta_1 x + \theta_2) + \theta_3$$



Squared-error loss

$$\mathcal{E}(\boldsymbol{\theta}) = \frac{1}{2} \sum_{i=1}^n [f(x_i, \boldsymbol{\theta}) - y_i]^2$$

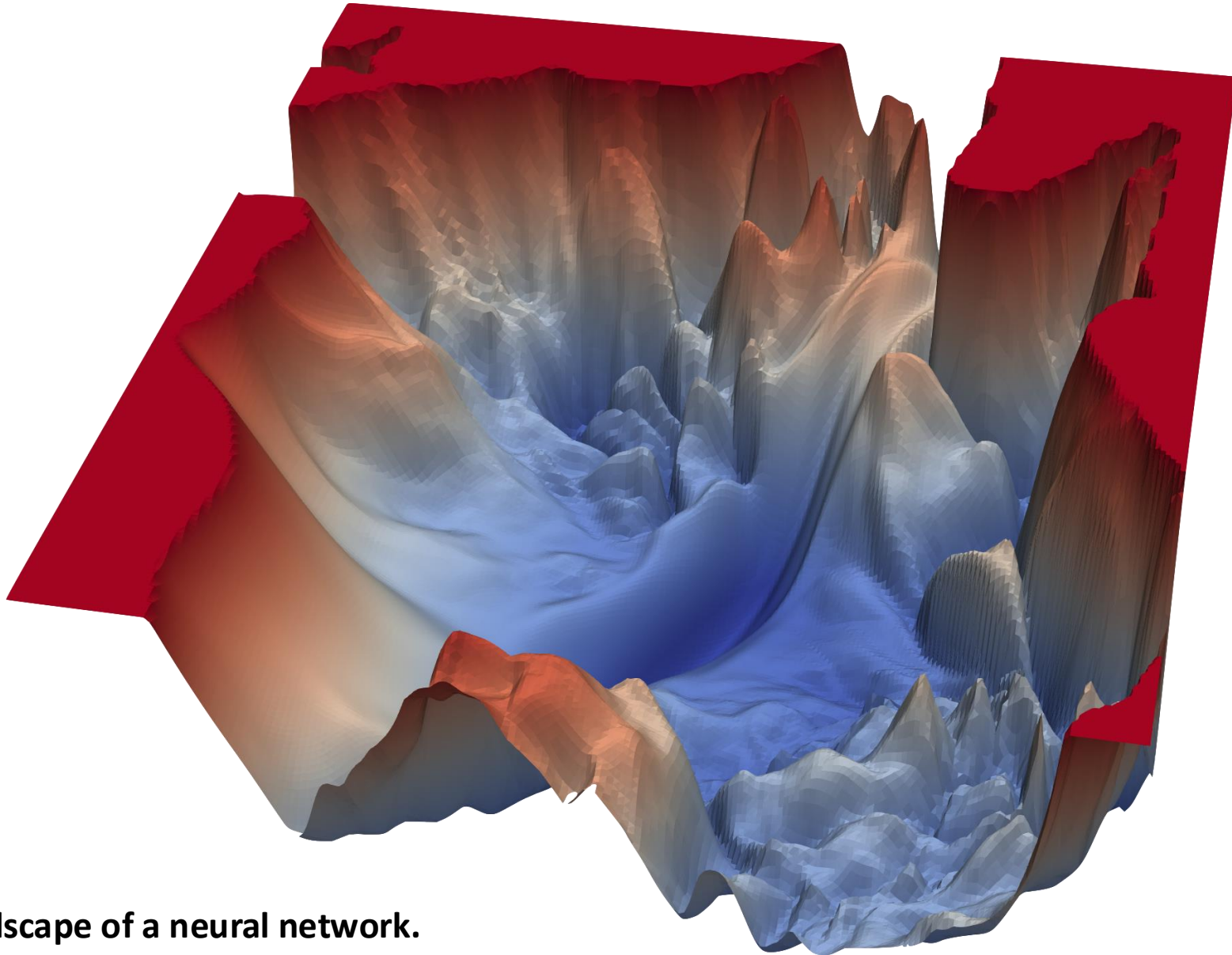
$$e_i = f(x_i, \boldsymbol{\theta}) - y_i$$

Stationary-point condition

$$\frac{\partial \mathcal{E}}{\partial \theta_j} = \sum_{i=1}^n e_i \frac{\partial f(x_i, \boldsymbol{\theta})}{\partial \theta_j} = 0 \quad \forall j$$

- A **stationary point** is a location where the slope is zero in every parameter direction.
- Zero gradient means “no local downhill direction,” not “lowest error everywhere.”
- **A stationary point does not guarantee a global minimum.**

Nonlinear Loss Landscapes



This is a loss landscape of a neural network.

Nonlinear Fitting with SciPy

scipy.optimize.

curve_fit

`curve_fit(f, xdata, ydata, p0=None, sigma=None, absolute_sigma=False, check_finite=None, bounds=(-inf, inf), method=None, jac=None, *, full_output=False, nan_policy=None, **kwargs)` [\[source\]](#)

Use non-linear least squares to fit a function, `f`, to data.

Assumes `ydata = f(xdata, *params) + eps`.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

def func(x, a, b, c):
    return a * np.exp(-b * x) + c
```

$$f(x, \theta) = ae^{-bx} + c, \quad \theta = \begin{bmatrix} a \\ b \\ c \end{bmatrix}$$

Define the data with some noise

```
X = np.linspace(0, 4, 50)
y_clean = func(X, 2.5, 1.3, 0.5)

rng = np.random.default_rng(seed=26)
noise = 0.2 * rng.normal(size=X.size)

y = y_clean + noise
```

```
popt, pcov = curve_fit(func, X, y)

print(popt)

[2.38353999  1.15045256  0.46503279]
```

Returns:

popt : *array*

Optimal values for the parameters so that the sum of the squared residuals of `f(xdata, *popt) - ydata` is minimized.

pcov : *2-D array*

The estimated approximate covariance of `popt`. The diagonals provide

True and Fitted Curves

```
y_true = func(X, 2.5, 1.3, 0.5)

fig, ax = plt.subplots(figsize=(5, 5))

ax.scatter(X, y, color="#4D4D4D", alpha=0.7, label="Data")

ax.plot(X, y_true, color="#0072B2", lw=2.5, label="True")

ax.plot(X, func(X, *popt), "--", color="#D55E00", lw=2.5, label="Fitted")

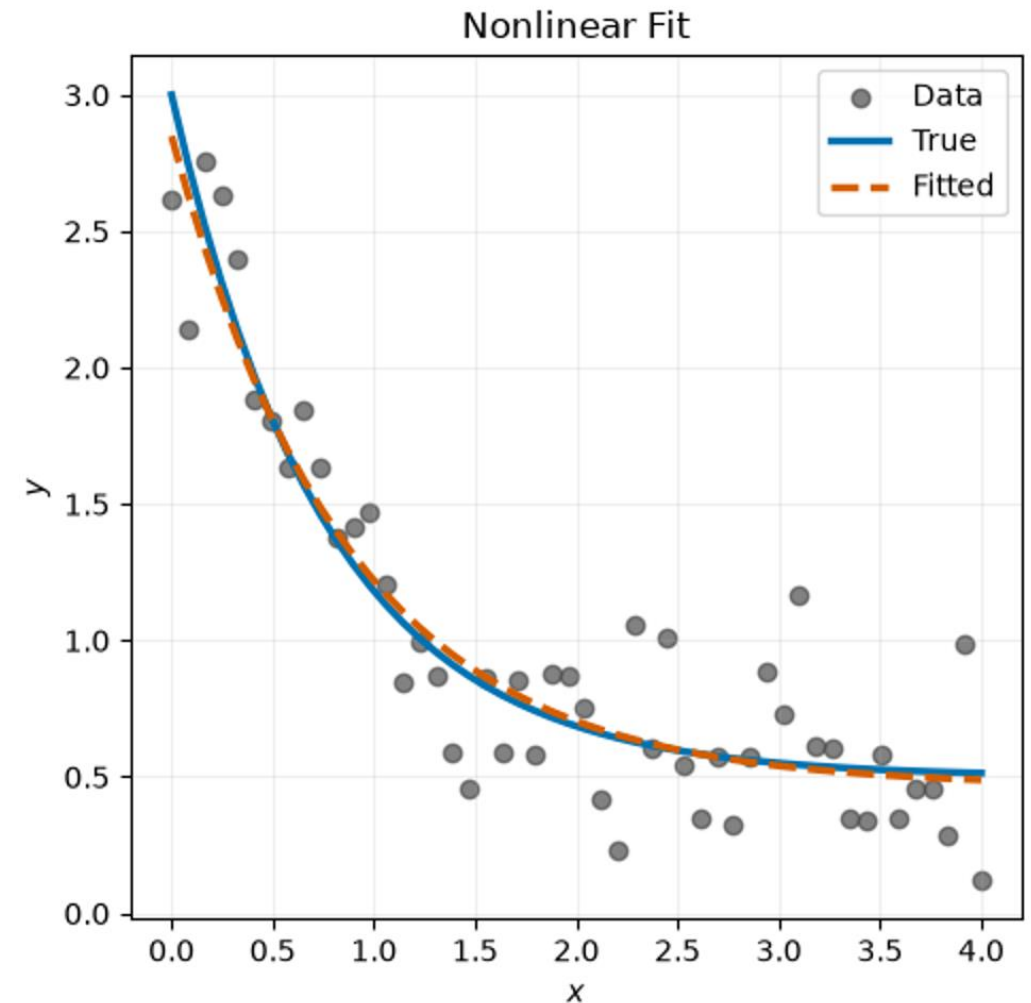
ax.set(xlabel="$x$", ylabel="$y$", title="Nonlinear Fit")

ax.legend()

ax.grid(alpha=0.2)

fig.tight_layout()

plt.show()
```



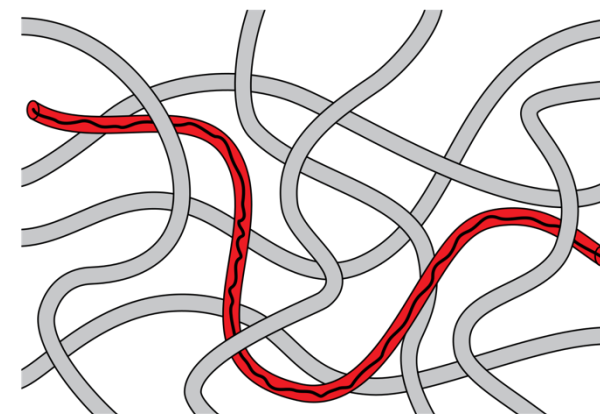
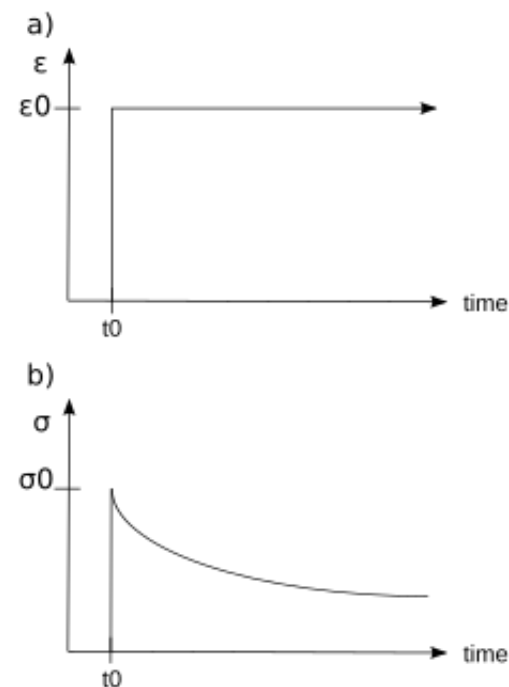
Modeling Polymer Stress Relaxation

How quickly does a polymer “forget” a deformation?

When a polymer is suddenly stretched and held at constant strain, its stress gradually decreases as the molecular chains rearrange.

$$\hat{G}(t; \boldsymbol{\theta}) = G_0 \exp\left(-\frac{t}{\tau}\right) \quad \boldsymbol{\theta} = \begin{bmatrix} G_0 \\ \tau \end{bmatrix}$$

- G_0 is the initial relaxation modulus (stiffness immediately after deformation)
- τ is the characteristic relaxation time
- At $t = \tau$, the modulus has fallen to $(\frac{G_0}{e} \approx 0.37G_0)$

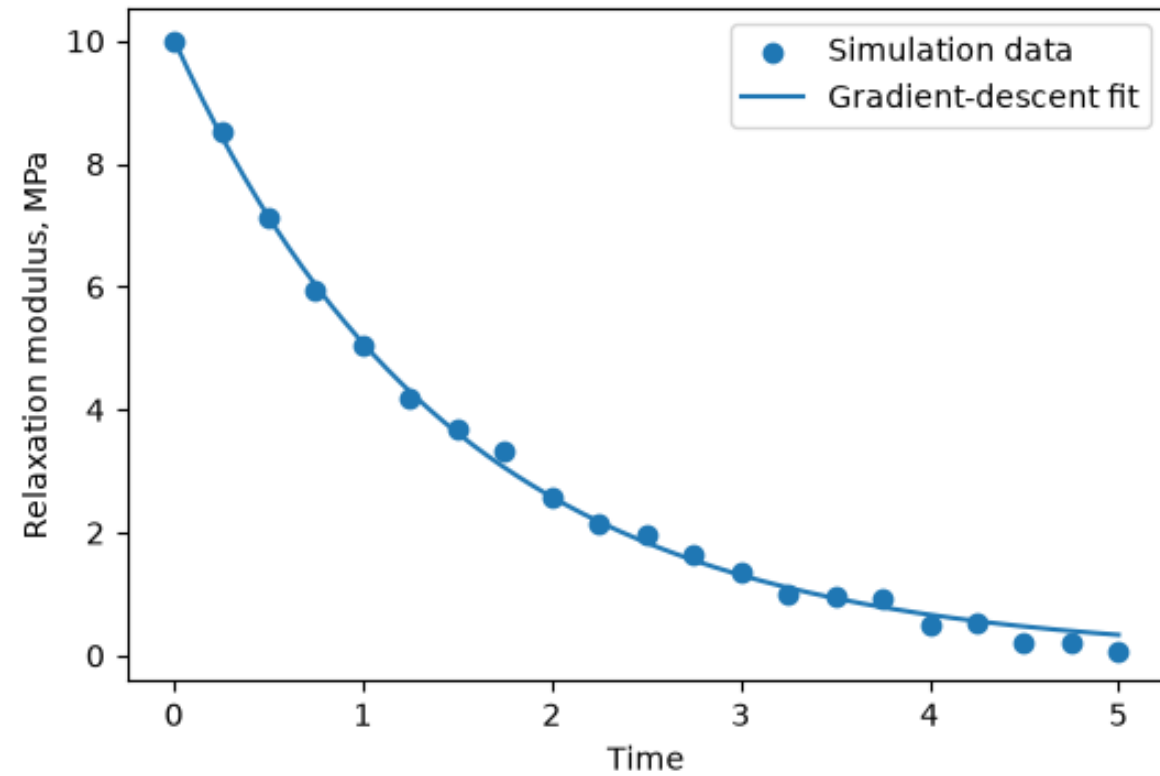


$$J(\boldsymbol{\theta}) = \frac{1}{2n} \sum_{i=1}^n \left[\hat{G}(t_i; \boldsymbol{\theta}) - G_i \right]^2$$

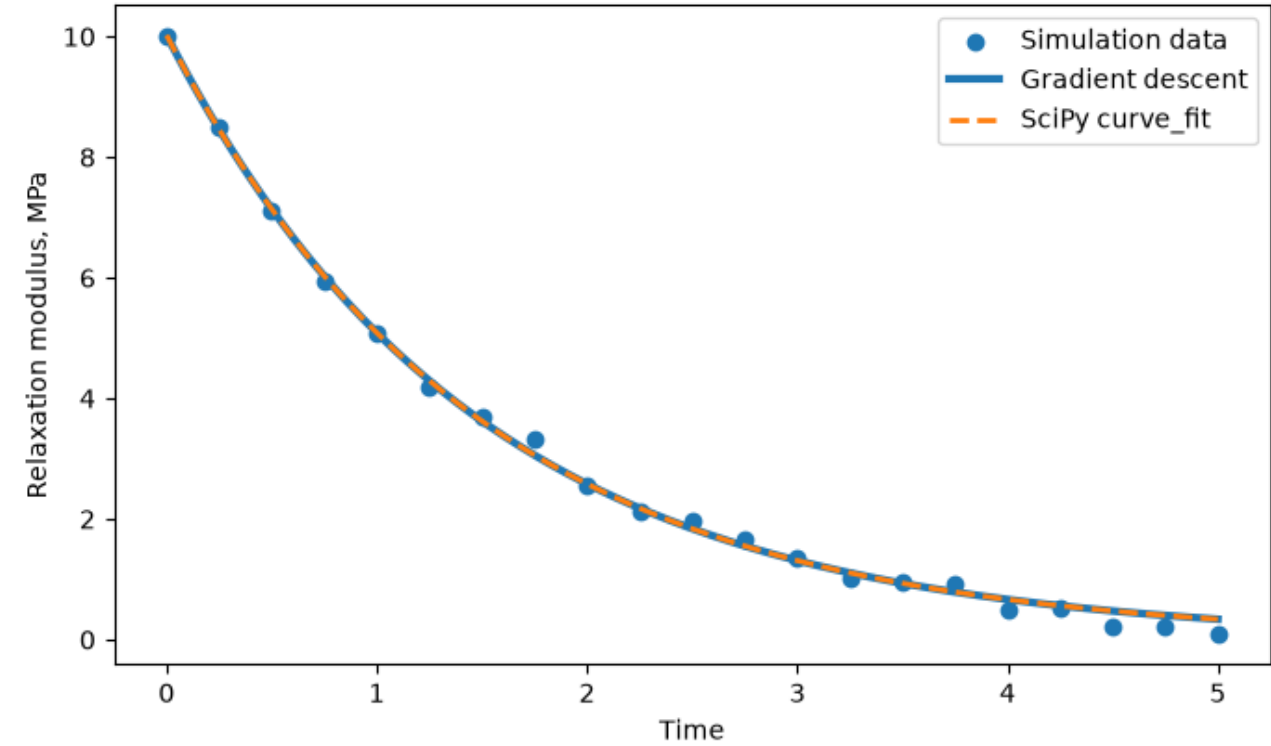
$$\frac{\partial \hat{G}}{\partial G_0} = \exp\left(-\frac{t}{\tau}\right) \quad \frac{\partial \hat{G}}{\partial \tau} = G_0 \exp\left(-\frac{t}{\tau}\right) \frac{t}{\tau^2}$$

Comparing Two Fitting Methods

Maxwell Stress Relaxation



Nonlinear Fit of Polymer Stress Relaxation



Interactive Python Time

06_nonlinear_curve_fit.ipynb